

Local AI Study Pipeline - Master Sheet v2.1

Project name: Local AI Study Pipeline

Optional app name: StudyForge

Version: v2.1

Core principle: AI builds the map. I still walk the terrain.

Table of Contents

1. Main Goal
2. Core Philosophy
3. Big-Picture Architecture
4. Long-Term Vision
5. Tool Roles
6. Recommended Folder Structure
7. GUI Concept
8. Pipeline View
9. Minimum Viable App
10. Later App Features
11. Local Digestion Prompt
12. Local Digest Output Template
13. Intermediate Auditor Prompt
14. Intermediate Audit Output Template
15. Final Auditor Prompt
16. Final Audit Output Template

17. Active Recall Prompt
 18. Homework Helper Prompt
 19. Autonomous Agent Roadmap
 20. Agent Safety Rules
 21. Quality Checklist
 22. Mistakes Log Template
 23. Weak Points Tracker
 24. Weekly Study Routine
 25. Best Use Cases
 26. Technical Stack Ideas
 27. Development Priority
 28. Final Core Idea
 29. Source Packet Standard
 30. Claim ID and Verification System
 31. Trust Levels for Study Materials
 32. Definition of Done for Each Pipeline Stage
 33. Failure Modes and Recovery Rules
 34. File Versioning and Audit History
 35. Anti-Slop Rules
 36. MVP Implementation Spec Snapshot
 37. Recommended Database Tables
 38. Final Build Principle
-

1. Main Goal

Build a local-first AI study system that helps me digest textbooks, study books, homework, notes, slides, and PDFs deeply and accurately.

The system should:

- process study material slowly and thoroughly
- generate structured notes
- make flashcards
- make practice questions
- identify weak points
- track mistakes
- audit AI-generated study material for accuracy
- help me prepare for tests and homework
- eventually become an autonomous local study agent

The purpose is not to blindly outsource homework.

The purpose is to learn faster, understand better, catch mistakes earlier, and study more actively.

2. Core Philosophy

**The AI stack builds the map.
I still walk the terrain.**

That means:

- AI summarizes.
- AI audits.
- AI quizzes.
- AI explains.
- AI tracks weak points.
- AI prepares study material.

But:

- I practice.
- I answer.
- I correct mistakes.
- I build understanding.

The system should help me become better, not replace my brain.

3. Big-Picture Architecture

Textbook / PDF / Notes / Homework

->

Local LLM through LM Studio

Slow digestion, first-pass study material

->

Intermediate Auditor

Google AI Studio / Gemini API

Checks local digest against source material

->

Final Auditor

ChatGPT / strongest available reasoning model

Validates everything, fixes weak points, tutors me

->

Study Mode

Quizzes, flashcards, practice, mistake tracking

->

Me

Active recall, homework practice, exam prep

The architecture is intentionally layered. No single model is treated as automatically trustworthy.

4. Long-Term Vision

The project should evolve through three stages.

Stage 1 - Controlled Pipeline

This is the first version.

The app does not need a fully autonomous agent yet. The app controls the workflow:

1. I upload source material.
2. I choose a chapter or section.
3. The app chunks the source.
4. The app sends chunks to LM Studio.
5. The local model creates a digest.
6. The app saves the output.
7. I send the digest to Gemini for audit.
8. I send source + digest + audit to ChatGPT for final review.
9. The app stores final study materials.

This stage should be reliable, boring, and useful.

Boring is good.

Stage 2 - Semi-Agentive Pipeline

The system starts making small decisions.

Examples:

- detect unclear sections automatically
- flag uncertain claims
- decide which sections need audit
- generate extra practice when confidence is low
- update weak-points tracker automatically
- suggest what I should study next
- compare my quiz answers against the final study guide

The app still controls the structure. The local model helps inside controlled boundaries.

Stage 3 - Autonomous Local Study Agent

This is the ultimate goal.

The local agent should eventually be able to:

1. Read a syllabus.
2. Understand the course structure.
3. Ingest textbooks, notes, slides, and homework.
4. Build a course knowledge base.
5. Digest chapters automatically.
6. Create study guides.
7. Detect gaps in understanding.
8. Generate quizzes.
9. Track my mistakes.
10. Schedule review sessions.
11. Ask for stronger AI audits when needed.
12. Maintain a long-term memory of my weak points.
13. Prepare me for upcoming tests.

The final version should feel like a private local tutor-librarian that understands my course.

It should not just answer questions. It should manage the study process.

5. Tool Roles

LM Studio

Role:

- local model runtime
- OpenAI-compatible API server
- local inference backend

LM Studio is not the full agent. LM Studio is the local brain.

The app provides:

- workflow
- memory
- file management
- chunking
- tool use
- progress tracking
- retry logic
- auditing steps
- GUI

Important distinction:

LM Studio provides model inference. StudyForge provides the workflow, source tracking, memory, audit structure, file management, and study logic.

Local LLM

Best for:

- slow chapter digestion
- private document analysis
- first-pass summaries
- glossary creation
- flashcards
- practice questions
- uncertainty lists
- basic tutoring
- cheap repeated tasks

The local model does not need to be perfect. Its job is to create the first structured draft.

Google AI Studio / Gemini API

Role: intermediate auditor.

Best for:

- checking local notes against source excerpts
- finding missing concepts
- spotting hallucinations
- reviewing formulas

- finding weak explanations
- flagging unsupported claims

Gemini is the second layer of quality control.

ChatGPT / Strongest Reasoning Model

Role:

- final auditor
- tutor
- examiner
- study coach

Best for:

- validating the Gemini audit
- checking disputed claims
- rewriting weak explanations
- creating final study guides
- quizzing me
- helping with homework as a tutor
- identifying exam traps

The strongest model should not waste time doing all the boring digestion from scratch. It should be used for high-value final verification and tutoring.

6. Recommended Folder Structure

Use one main folder per course.

```
Course_Name/
|
|-- 00_Master/
|   |-- study_pipeline_master_sheet.md
|   |-- course_profile.md
|   |-- course_syllabus.md
|   |-- exam_targets.md
|   |-- prompt_templates.md
|   |-- pipeline_settings.json
|
|-- 01_Source_Material/
|   |-- textbook/
|   |-- lecture_notes/
|   |-- homework/
```

```
| |-- teacher_slides/
| |-- extra_readings/
|
|-- 02_Extracted_Text/
| |-- chapter_01_chunks/
| |-- chapter_02_chunks/
| |-- extraction_logs/
|
|-- 03_Local_Digests/
| |-- chapter_01/
| | |-- digest_v001.md
| | |-- digest_v002.md
| | |-- latest.md
| |-- chapter_02/
|
|-- 04_Intermediate_Audits/
| |-- chapter_01/
| | |-- gemini_audit_v001.md
| | |-- latest.md
|
|-- 05_Final_Audits/
| |-- chapter_01/
| | |-- final_audit_v001.md
| | |-- latest.md
|
|-- 06_Study_Outputs/
| |-- study_guides/
| |-- flashcards/
| |-- formula_sheets/
| |-- glossaries/
| |-- practice_quizzes/
| |-- mock_tests/
|
|-- 07_My_Work/
| |-- solved_practice/
| |-- mistakes_log.md
| |-- weak_points.md
|
|-- 08_App_Data/
| |-- database.sqlite
| |-- vector_store/
| |-- job_logs/
| |-- exports/
```


Why add 02_Extracted_Text?

PDF extraction quality is one of the biggest risks in this project. The app should always preserve:

- original source
- extracted text
- chunked text
- local digest
- audit outputs
- final study output

Bad extraction can poison the entire pipeline. The extracted-text layer makes errors easier to detect and fix.

7. GUI Concept

The GUI should be built around study jobs, not random chat.

Chat is useful, but the main interface should be a pipeline dashboard.

Main Sidebar

- Dashboard
- Courses
- Sources
- Pipelines
- Digests
- Audits
- Study Guides
- Flashcards
- Quizzes
- Mistakes Log
- Weak Points
- Settings

Dashboard

The dashboard should show:

- active course
- current chapter
- local digest status
- intermediate audit status
- final audit status
- flashcards due

- weak points
- mistakes to review
- next recommended action

Example:

Course: Biology

Chapter: 4 - Cell Respiration

Local digest: Complete

Gemini audit: Complete

Final audit: Pending

Weak points: 5

Flashcards due: 24

Next action: Run final audit

8. Pipeline View

The main workflow should look like this:

[Upload Source]

->

[Extract / Chunk Source]

->

[Local Digest]

->

[Intermediate Audit]

->

[Final Audit]

->

[Study Pack]

->

[Quiz / Review]

Each stage should show:

- status
- model used
- prompt used
- input files
- output files
- warnings
- run button
- review button

- export button

Possible statuses:

- Not started
 - Running
 - Needs review
 - Complete
 - Failed
 - Needs stronger audit
-

9. Minimum Viable App

Do not build the giant autonomous system first.

Build the useful loop first.

MVP Features

1. Course selector
2. Source upload
3. Chapter/section selector
4. Local digest button
5. Save digest as Markdown
6. Paste/import Gemini audit
7. Paste/import final audit
8. Generate flashcards
9. Generate practice questions
10. Track weak points
11. Track mistakes

The first working version should answer this:

Can I upload a chapter, digest it locally, audit it, and study from it?

If yes, the project is already useful.

Even smaller MVP 0.1

The first practical build can be smaller:

1. Create/select course folder
2. Add PDF/source file
3. Select page range or chapter
4. Extract text
5. Send chunks to LM Studio
6. Save local digest as Markdown

7. Show digest in GUI
8. Let user paste Gemini audit
9. Let user paste final audit
10. Save everything in the right folders

No flashcards yet. No quizzes yet. No vector database yet.

First test:

Can StudyForge reliably take Chapter 1 and produce a saved digest I can audit?

10. Later App Features

After the MVP works, add:

- automatic chunking
 - PDF page references
 - source citation extraction
 - vector database search
 - chapter progress tracking
 - flashcard scheduling
 - mistake review scheduling
 - automatic weak-point detection
 - automatic audit requests
 - multi-model comparison
 - autonomous study planning
-

11. Local Digestion Prompt

Use this with the local LLM through LM Studio.

You are my local study assistant.

Your job is to digest the source material slowly and carefully.

Do not over-compress.

Do not skip difficult parts.

Do not invent facts.

Work section by section.

For each section, produce:

1. Section title
2. Plain-language explanation

3. Key definitions
4. Important formulas, rules, methods, or principles
5. Step-by-step explanation of any procedures
6. Worked examples from the source
7. New examples that match the style of the source
8. Common mistakes or traps
9. Connections to earlier sections
10. Flashcards
11. Practice questions
12. Things I may need to memorize
13. Things I need to truly understand
14. Uncertain claims or places where you might be wrong
15. Page, chapter, or section references whenever possible

Rules:

- If the source does not clearly support something, mark it as uncertain.
 - Keep references to the source.
 - Prefer clarity over shortness.
 - At the end of each chapter, create:
 - chapter summary
 - glossary
 - concept map in text form
 - formula/method sheet
 - practice quiz
 - weak-points checklist
-

12. Local Digest Output Template

Chapter X Local Digest

Source Info

Course:

Book/PDF:

Chapter:

Sections:

Pages:

Source packet IDs:

Digest version:

Model used:

Date:

Big Picture

Explain what this chapter is about and why it matters.

Section-by-Section Notes

Section X.X - Title

Simple Explanation

Key Definitions

Important Rules / Formulas / Methods

Worked Examples

New Examples

Common Mistakes

Connections to Earlier Material

Flashcards

Practice Questions

Uncertain Claims

Source References

Chapter Summary

Glossary

Formula / Method Sheet

Practice Quiz

Weak Points to Review

Questions to Ask a Stronger AI

13. Intermediate Auditor Prompt

Use this with Google AI Studio / Gemini API.

You are the intermediate auditor for my study pipeline.

I will provide:

1. Source material excerpts
2. A local LLM digest of that material

Your job is to audit the local digest against the source.

Check for:

- incorrect claims
- hallucinations
- missing key concepts
- weak explanations
- misleading simplifications
- wrong formulas
- missing assumptions
- skipped examples
- bad practice questions
- unsupported claims
- likely test traps

For every issue, return:

1. The original claim from the local digest
2. Verdict:
 - Correct
 - Incomplete
 - Misleading
 - Wrong
 - Not verified by source
3. Corrected version
4. Source reference
5. Why this matters for homework, quizzes, or exams

Do not guess.

If the source does not support a claim, say "not verified."

14. Intermediate Audit Output Template

Chapter X Intermediate Audit

Overall Verdict

Accuracy:

Completeness:

Clarity:

Usefulness for homework/tests:

Major Issues

Issue 1

Original claim:

Verdict:

Correction:

Source reference:

Why it matters:

Missing Concepts

Weak Explanations

Wrong or Risky Formulas

Unsupported Claims

Exam / Homework Traps

Recommended Fixes

Questions for Final Auditor

15. Final Auditor Prompt

Use this with the strongest available AI.

You are the final auditor and tutor for my study pipeline.

I will provide:

1. Source excerpts
2. The local LLM digest
3. The intermediate AI audit
4. My course goals, homework goals, or exam goals if available

Your job:

- verify whether the intermediate audit is correct
- catch remaining errors
- identify missing concepts
- correct weak or misleading explanations
- explain difficult ideas clearly
- decide what matters most for homework/tests
- produce a final study guide
- create active recall questions
- quiz me one question at a time afterward

Rules:

- Do not blindly trust the local digest.
 - Do not blindly trust the intermediate audit.
 - Use the source excerpts as the highest authority.
 - If something is not supported by the source, say "not verified."
 - Do not give me final homework answers immediately.
 - Prefer hints, explanations, and checks of my own work.
-

16. Final Audit Output Template

Chapter X Final Audit

Final Verdict

Accuracy:

Completeness:

Exam usefulness:

Biggest risks:

Corrections That Matter Most

Correction 1

Problem:

Correct version:

Why it matters:

Source support:

Missing Concepts

Better Explanations

Must-Memorize List

Must-Understand List

Formula / Method Sheet

Exam Traps

Practice Problems

Active Recall Questions

Final Study Checklist

My Weak Points

Next Study Session Plan

17. Active Recall Prompt

Use this after the final audit.

Quiz me on this chapter one question at a time.

Rules:

- Ask only one question.
 - Wait for my answer.
 - Do not reveal the answer immediately.
 - If I am wrong, give a hint first.
 - If I am still stuck, explain the answer step by step.
 - Track my weak points.
 - After 10 questions, summarize what I know well and what I need to review.
-

18. Homework Helper Prompt

Use this to avoid accidentally outsourcing the homework.

Help me with this homework problem as a tutor.

Rules:

- Do not give the final answer immediately.
 - Ask me what I think the first step is.
 - Give hints before giving solutions.
 - Check my reasoning.
 - Explain mistakes clearly.
 - After I solve it, give me a similar practice problem.
-

19. Autonomous Agent Roadmap

The autonomous agent should be built carefully.

The goal is not a chaotic free-roaming agent. The goal is a reliable study agent with clear permissions, memory, and review checkpoints.

Phase A - Deterministic Pipelines

The app controls everything:

- digest chapter
- audit chapter
- generate flashcards
- generate quiz
- update weak points
- export study guide

The model only responds to fixed prompts.

Phase B - Tool-Assisted Agent

The local model can use tools, but only inside safe boundaries.

Possible tools:

- read_source_file
- search_course_notes
- write_digest
- create_flashcards
- create_quiz
- update_weak_points
- log_mistake
- request_intermediate_audit
- mark_claim_uncertain

The app still approves important steps.

Phase C - Planner Agent

The system can plan study sessions.

Example:

The exam is in 5 days.

Chapter 3 has many weak points.

The student failed 6/10 questions on formulas.

Schedule formula review and generate more practice.

The agent can suggest what to study next. The user still confirms major actions.

Phase D - Autonomous Study Manager

Ultimate goal.

The agent can:

- read the syllabus
- ingest course material
- digest new chapters automatically
- create study packs
- audit weak areas
- quiz me regularly
- track my mistakes
- recommend reviews
- prepare mock tests
- escalate uncertain material to stronger AI
- maintain long-term course memory

This version should feel like:

- a local private tutor
- a course librarian
- a study planner
- an examiner
- a mistake tracker

All in one.

20. Agent Safety Rules

The autonomous system should follow these rules:

- Never silently trust its own output.
- Always preserve source references.
- Always mark uncertainty.
- Always keep audit history.
- Always separate source text from AI interpretation.
- Always let me review final study guides.
- Never fabricate page references.
- Never give final homework answers before tutoring.
- Never delete source files.
- Never overwrite final notes without version history.

The agent should be powerful, but not reckless.

No goblin autopilot.

21. Quality Checklist

Before trusting a study digest, check:

- ☐ Does it cite the chapter/section/page?
- ☐ Are formulas copied correctly?
- ☐ Are definitions precise?
- ☐ Are examples faithful to the source?
- ☐ Did it mark uncertain claims?
- ☐ Did the intermediate auditor verify the claims?
- ☐ Did the final auditor check the auditor?
- ☐ Did I test myself with active recall?
- ☐ Did I solve practice problems myself?
- ☐ Did I write down my mistakes?

22. Mistakes Log Template

Mistakes Log

Mistake

Date:

Course:

Chapter:

Problem/topic:

What I got wrong:

Why I got it wrong:

Correct explanation:

How to avoid it next time:

Similar practice problem:

Review status:

Review statuses:

- New
- Reviewed once
- Still weak
- Mastered

23. Weak Points Tracker

Weak Points

Concept

Chapter/section:

Confidence level:

Why it feels hard:

What I need to review:

Practice needed:

Status:

Confidence levels:

- 1 = I do not understand it
 - 2 = I recognize it but cannot use it
 - 3 = I can follow examples
 - 4 = I can solve normal problems
 - 5 = I can explain it and solve harder problems
-

24. Weekly Study Routine

Before studying

1. Pick one chapter or topic.
2. Run local digestion.
3. Check uncertain claims.
4. Send digest + source excerpts to intermediate auditor.
5. Send source + digest + audit to final auditor.
6. Save final corrected guide.

During studying

1. Read final corrected guide.
2. Do active recall questions.
3. Solve practice problems.
4. Log mistakes.
5. Review weak points.

Before a test

1. Review must-memorize list.
 2. Review must-understand list.
 3. Redo missed problems.
 4. Ask final auditor for a mock test.
 5. Grade the mock test.
 6. Study only the weak areas.
-

25. Best Use Cases

This pipeline is best for:

- textbooks
- study books
- lecture notes
- PDFs
- homework review
- exam prep
- glossaries

- flashcards
- practice quizzes
- step-by-step understanding
- mock tests
- mistake tracking

It is not ideal for:

- blindly solving graded work
 - replacing actual practice
 - trusting summaries without source checks
 - letting AI decide what the teacher cares about without syllabus/notes
 - building a giant app before studying
-

26. Technical Stack Ideas

Simple MVP

Frontend: Streamlit

Backend: Python

Storage: Markdown files + SQLite

Local model: LM Studio OpenAI-compatible API

Retrieval: ChromaDB or LanceDB later

Polished App

Frontend: Next.js, React, Tailwind, shadcn/ui

Backend: FastAPI

Storage: SQLite first, Postgres later

Vector search: ChromaDB, LanceDB, Qdrant

Desktop wrapper: Tauri

27. Development Priority

Build in this order:

1. Folder structure
2. Local digest script
3. Markdown output
4. Basic GUI
5. Gemini audit import/export
6. Final audit import/export
7. Flashcards
8. Quizzes

9. Mistakes log
10. Weak-points tracker
11. Source citations
12. Vector search
13. Semi-agentic tools
14. Autonomous planner
15. Fully autonomous study manager

Do not start with the autonomous monster.

Earn the monster.

28. Final Core Idea

The first version should be simple:

source material

-> local digest

-> AI audit

-> final study guide

-> active recall

-> mistake tracking

The ultimate version should be autonomous:

course material

-> local study agent

-> automatic digestion

-> automatic audits

-> adaptive quizzes

-> weak-point tracking

-> exam prep plan

The dream is a private AI study system that gets better the more I use it.

Not just a chatbot.

A study engine.

29. Source Packet Standard

Before any AI model digests or audits source material, the app should create a source packet.

A source packet is a clean, traceable unit of study material extracted from the original source.

Each source packet should include:

- course name
- source file name
- source type: textbook, slides, notes, homework, syllabus, other
- chapter or section
- page range, when available
- chunk ID
- extracted text
- extraction method
- OCR used: yes/no
- extraction warnings
- date created

Purpose

The source packet allows every digest, audit, flashcard, quiz, and final study guide to connect back to the original material.

The system should never rely only on vague references like “the textbook says.” It should preserve exact source links through page numbers, section names, or chunk IDs.

If page numbers are not available, the app should use chunk IDs instead.

Source Packet Template

Source Packet

Course:

Source file:

Source type:

Chapter:

Section:

Page range:

Chunk ID:

Extraction method:

OCR used:

Extraction warnings:

Date created:

Extracted Text

[Exact extracted text here]

Notes

- Any formatting problems

- Missing images/tables
 - Page number uncertainty
 - OCR uncertainty
-

30. Claim ID and Verification System

The app should treat important AI-generated statements as claims that can be checked.

A claim can be:

- a definition
- a formula
- a procedure
- a rule
- a worked-example explanation
- an exam warning
- a conceptual interpretation
- a connection between ideas

Each important claim should receive a unique ID.

Suggested format:

CLAIM-COURSE-CHAPTER-NUMBER

Example:

CLAIM-ECA1010-CH04-003

Claim: Productivity is usually measured as output per hour worked.

Source: Chapter 4, p. 92

Confidence: Medium

Verification status: Pending

Each claim should include:

- claim ID
- claim text
- claim type
- claimed source reference
- confidence level
- verification status
- audit notes

Verification statuses:

- Pending

- Verified
- Partially verified
- Incorrect
- Not supported by source
- Needs final auditor
- User validated

Purpose

This prevents the system from treating a polished paragraph as automatically trustworthy.

The goal is not just to generate notes. The goal is to know which notes are actually supported.

31. Trust Levels for Study Materials

Every study output should have a trust level.

Trust level	Meaning
Draft	Generated by the local model. Useful, but not fully trusted.
Intermediate audited	Checked by Gemini or another intermediate auditor.
Final reviewed	Checked by the strongest available reasoning model.
User validated	Tested through active recall, homework practice, or correction by the user.
Needs review	Contains uncertain, unsupported, contradictory, or weak material.

The dashboard should show trust levels clearly.

Example:

Course: ECA1010

Chapter: 4 - Productivity and Living Standards

Local digest: Complete

Trust level: Draft

Intermediate audit: Complete

Trust level: Intermediate audited

Final audit: Pending

Trust level: Needs review

Flashcards: Generated
Trust level: Based on draft only

The system should avoid presenting draft material as final truth.

32. Definition of Done for Each Pipeline Stage

Each stage of the pipeline should have a clear definition of done.

Stage	Done when...
Local Digest	Source packets were created; chunks were processed; digest was saved as Markdown; uncertain claims were marked; source references or chunk IDs were included; job log was saved.
Intermediate Audit	The digest was checked against source packets; incorrect claims were identified; unsupported claims were flagged; missing concepts were listed; risky formulas or explanations were reviewed; recommended fixes were saved.
Final Audit	The intermediate audit was reviewed; remaining errors were corrected; final study guide was created; must-memorize and must-understand lists were created; exam traps were identified; active recall questions were generated; unresolved uncertainties were preserved.
Study Pack	Flashcards were generated; practice questions were generated; weak points were initialized or updated; mistakes log is ready for use; review checklist was created.
Review Session	User answered active recall questions; mistakes were logged; weak points were updated; next study action was chosen.

A chapter should not be marked as fully complete until the final audit and at least one active recall session are done.

33. Failure Modes and Recovery Rules

The app should expect problems and handle them clearly.

Problem	What the app should do
Bad PDF extraction	Warn the user, save extraction log, mark source packet as low confidence.
Missing page numbers	Use chunk IDs instead of fabricated page numbers.
OCR uncertainty	Mark text as OCR-derived and flag the packet for review.
Model returns garbage	Save failed output, mark job as failed, allow retry.
Model output is incomplete	Mark digest as incomplete and allow continuation from last chunk.
Audit contradicts digest	Mark the section as needs final review.
Final audit finds unsupported claims	Remove them or label them as not verified.
User edits final guide	Save a new version instead of overwriting.
Source file changes	Mark linked digests/audits as potentially stale.
Prompt template changes	Save the new prompt version and record which version each job used.

The system should never pretend that a broken pipeline stage succeeded.

34. File Versioning and Audit History

The app should preserve old versions instead of overwriting important files.

Recommended pattern:

03_Local_Digests/

```
-- chapter_01/
|  -- digest_v001.md
|  -- digest_v002.md
|  -- latest.md
```

04_Intermediate_Audits/

```
-- chapter_01/
|  -- gemini_audit_v001.md
|  -- gemini_audit_v002.md
|  -- latest.md
```

05_Final_Audits/

```
-- chapter_01/
|  -- final_audit_v001.md
```

```
| |-- final_audit_v002.md
| |-- latest.md
```

Each version should record:

- date created
- model used
- prompt version
- source packet IDs
- input files
- output file
- known warnings
- trust level

Why this matters

Versioning prevents confusion like:

Which audit fixed this mistake?

or:

Did this final guide come from the corrected digest or the old one?

Audit history makes the study system trustworthy over time.

35. Anti-Slop Rules

The system should prefer reliable, traceable, and useful study material over impressive-looking output.

Rules:

- Prefer incomplete but verified notes over complete but unreliable notes.
- Never hide uncertainty.
- Never invent page numbers.
- Never treat citations as proof unless the cited source actually supports the claim.
- Never generate final flashcards from unreviewed material without marking them as draft.
- Never overwrite corrected notes without saving a previous version.
- Never mix source text and AI interpretation without labels.
- Never mark a chapter as mastered just because notes were generated.
- Do not optimize only for speed.
- Do not create beautiful summaries that are too vague to help with homework.
- Every final study guide should help the user answer questions, solve problems, and explain concepts.

Core principle:

A shorter verified study guide is better than a long hallucinated one.

36. MVP Implementation Spec Snapshot

The master sheet is the north star. The MVP spec is the hammer.

The first implementation should be brutally small.

MVP Goal

Build one working pipeline for:

- one course
- one PDF/source file
- one chapter or page range
- one local digest
- one imported intermediate audit
- one imported final audit

MVP Input

- course name
- source file
- chapter name or page range
- local model selection
- prompt template selection

MVP Output

- extracted text chunks
- local digest Markdown
- saved Gemini/intermediate audit
- saved final audit
- job log

MVP Done When

The system can answer:

Can I upload a chapter, digest it locally, audit it, and study from it?

If the answer is yes, the MVP is successful.

Suggested First CLI Command

Before building the GUI, make the backend work from the command line:

```
python studyforge.py digest --course ECA1010 --source textbook.pdf --pages 45-62
```


Expected output:

Course_Name/03_Local_Digests/chapter_01/digest_v001.md

Once this works, Streamlit becomes much easier.

37. Recommended Database Tables

Even if the main outputs are Markdown files, SQLite should track the pipeline.

Minimum useful tables:

- courses
- sources
- source_packets
- chunks
- jobs
- digests
- claims
- audits
- study_outputs
- flashcards
- quizzes
- mistakes
- weak_points

Most Important Table: Jobs

jobs

- id
- course_id
- job_type
- status
- source_id
- chapter
- model_used
- prompt_template
- prompt_version
- input_path
- output_path
- created_at
- completed_at
- error_message

The jobs table gives the dashboard almost for free.

Claims Table

claims

- id
- course_id
- chapter
- claim_id
- claim_text
- claim_type
- source_packet_id
- claimed_source_reference
- confidence_level
- verification_status
- audit_notes
- created_at
- updated_at

Weak Points Table

weak_points

- id
 - course_id
 - chapter
 - concept
 - confidence_level
 - why_hard
 - practice_needed
 - status
 - last_reviewed_at
 - next_review_at
-

38. Final Build Principle

Build the system in this order:

Reliability

- > Traceability
- > Study usefulness
- > Automation
- > Autonomy

Do not build the autonomous monster first.

A reliable boring pipeline beats a flashy unreliable agent.

The first version should feel like:

A clean study factory that turns source material into auditable study material.

The final version should feel like:

A private local tutor-librarian-examiner that knows my courses, remembers my mistakes, and helps me get better without replacing my thinking.

The goal is not AI autopilot.

The goal is guided mastery.